DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Software Engineering Course (CS302) • Mid-Semester Project Evaluation

AutoTestHub

Web Automation and Testing Platform using Selenium

MID-SEMESTER PROJECT REPORT • 50% MILESTONE • 20 MARKS

ACADEMIC & COURSE METADATA

Course Title: Software Engineering (CS302)

Academic Section: Section [X]

Project Group: Group [XX]

Submission Mode: Moodle Archive Package

Supervisor / Instructor: [Supervisor / Instructor Name]

Submission Date: October 19, 2026

PROJECT DEVELOPMENT TEAM

Student Name	Registration Number	Module Role
Shristi Neekhra	[Reg No: XXXXXXXX]	Integration Lead
Shifa Siddiqui	[Reg No: XXXXXXXX]	Backend & DB Lead
Shruti Gupta	[Reg No: XXXXXXXX]	Selenium Engine Lead
Shriya Agarwal	[Reg No: XXXXXXXX]	Frontend Lead

Table of Contents

Section	Section Title	Report Page
1	Cover Page & Academic Metadata	1
2	Problem Statement and Motivation	3
3	Project Objectives	3
4	Scope of the Project (Included, Excluded, Constraints)	3
5	Stakeholder Identification	4
6	Requirements Specification (Functional, Non-Functional, MoSCoW Matrix)	4
7	Requirements Model (Use-Case Model, Use-Case Specs, User Stories)	5
8	System Design (Architecture, Database ER, Sequence, Class Diagrams)	6–8
9	Technology Stack and Technical Justifications	9
10	Implementation Progress (Completed, In-Progress, Working Screenshots)	10
11	Testing Performed (Unit, Integration, API Matrices & Pass/Fail Logs)	11
12	Project-Management Status (Schedule, Milestones, Delayed Tasks, Risk Matrix)	12
13	Individual Contribution Table & Task Allocations	13
14	Challenges Faced and Solutions Attempted	13
15	Work Planned for Remaining Semester (Roadmap Sprints 4–8)	14
16	References & Academic Integrity AI Disclosure (Section 6 Compliance)	14

Executive Summary

This report documents the mid-semester evaluation of **AutoTestHub**, an automated web testing and test orchestration platform developed using Selenium 4, Node.js, Express, and MongoDB. By the mid-semester milestone, the project group has achieved **50% completion** (meeting the 40–60% evaluation requirement), featuring a fully functioning, verified end-to-end testing pipeline.

The platform allows testers to define browser actions (navigation, text input, button clicks, title assertions) declaratively in JSON, execute them in real Chrome instances (headless or windowed) via an asynchronous IPC execution service, capture automatic visual screenshots on pass or fail, and persist historical execution audits in a NoSQL document database. A suite of 16 automated tests (8 in-memory unit tests and 8 browser-driven integration tests) demonstrates a 100% pass rate.

2. Problem Statement and Motivation

Automated regression testing is essential for web continuous delivery. However, raw Selenium WebDriver scripts suffer from: (1) **Asynchronous Timing Flakiness** where dynamic DOM rendering causes false-negative failures; (2) **Driver Version Incompatibilities** where routine Chrome updates break local ChromeDriver binaries; (3) **Boilerplate Overhead** for setup, locators, and screenshot logging; and (4) **Disconnected Architectures** lacking unified REST APIs and database tracking.

Motivation: AutoTestHub resolves these pain points by offering a declarative JSON-driven testing platform unifying a web control desk, REST API gateway, child_process execution service, and headless Selenium engine into an integrated testing pipeline.

3. Project Objectives

1. **Declarative Test Ingestion:** Author browser tests via structured JSON schemas without imperative boilerplate.
2. **Decoupled Multi-Tier Topology:** Isolate frontend UI, Express gateway, child_process IPC, and browser workers.
3. **Zero-Config Driver Automation:** Leverage Selenium 4's native Selenium Manager for automatic ChromeDriver downloads.
4. **Explicit Dynamic Synchronization:** Support dynamic locators (ID, Name, CSS, XPath) and explicit WebDriverWait.
5. **Automated Evidence & Auditing:** Guarantee timestamped PNG screenshots on pass/fail and MongoDB persistence.
6. **Interactive Control Desk:** Deliver a web dashboard supporting live browser runs and simulation playback.

4. Scope of the Project

4.1 Features Included (Mid-Semester 50% Milestone)

JSON test definition parser, safe URL validator, core actions (`OPEN_URL` , `TYPE` , `CLICK` , `ASSERT_TITLE`), asynchronous child_process bridge, MongoDB schemas (`TestCase` , `TestResult`), REST endpoints (`POST /api/tests` , `GET /api/tests` , `POST /api/tests/:id/run`), automated screenshot capture, interactive Control Desk UI, and 16 automated tests.

4.2 Features Excluded & Constraints

Excluded from Mid-Sem: Multi-tenant role authentication, distributed Redis/BullMQ worker queues, cross-browser testing (Firefox/Edge), and visual pixel-diff regression testing (scheduled for final release).
Constraints: Requires Python 3.10+, Node.js 18+, Google Chrome, and local MongoDB. Single concurrent run per worker.

5. Stakeholder Identification

Stakeholder	Core Needs & Pain Points	Value Delivered by AutoTestHub
QA Automation Engineers	Fragile scripts, manual screenshot audits, locator maintenance.	Declarative JSON tests, explicit waits, dynamic locators, automated screenshots.
Software Developers	Need fast regression checks without local driver configuration.	Lightweight REST API endpoints and zero-config driver resolution.
Engineering Leads	Lack visibility into historical test regressions and team progress.	Persistent MongoDB execution logs, execution timings, and Control Desk.

6. Requirements Specification

6.1 Functional Requirements Matrix

ID	Requirement Description	Priority	Status
`FR-01`	Ingest and validate structured JSON test cases.	MUST	COMPLETED
`FR-02`	Enforce HTTP/HTTPS URL schemes; reject insecure protocols.	MUST	COMPLETED
`FR-03`	Dynamically resolve locator strategies (ID, Name, CSS, XPath).	MUST	COMPLETED
`FR-04`	Explicitly wait for element visibility and clear text on `TYPE`.	MUST	COMPLETED
`FR-05`	Explicitly wait for element clickability on `CLICK` actions.	MUST	COMPLETED
`FR-06`	Assert DOM page title with timeout and mismatch error handling.	MUST	COMPLETED
`FR-07`	Auto-capture timestamped PNG screenshots on pass and fail.	MUST	COMPLETED
`FR-08`	Guarantee `driver.quit()` execution to eliminate zombie processes.	MUST	COMPLETED
`FR-09`	Emit strict single-line JSON on stdout; diagnostic logs to stderr.	MUST	COMPLETED
`FR-10`	Express REST API gateway (`/api/tests` , `/api/health`).	MUST	COMPLETED
`FR-11`	Persist execution results and timings to MongoDB.	MUST	COMPLETED
`FR-12`	Interactive Selenium Control Desk UI with simulation mode.	SHOULD	COMPLETED

6.2 Non-Functional Requirements & Prioritization (MoSCoW)

Performance (NFR-01): Engine overhead < 250ms/action; in-memory validation < 5ms. **Reliability (NFR-02):** Exception isolation with fallback database failure records. **Portability (NFR-03):** macOS, Windows, Linux verified. **Security (NFR-04):** Strict protocol sanitization and `.env` credential isolation.
MoSCoW Audit: Must Have (100% Implemented); Should Have (100% Implemented); Could Have (Sprints 4–6); Won't Have (Deferred to Final).

7. Requirements Model

The requirements model defines actor interactions with AutoTestHub platform use cases, accompanied by agile user stories.

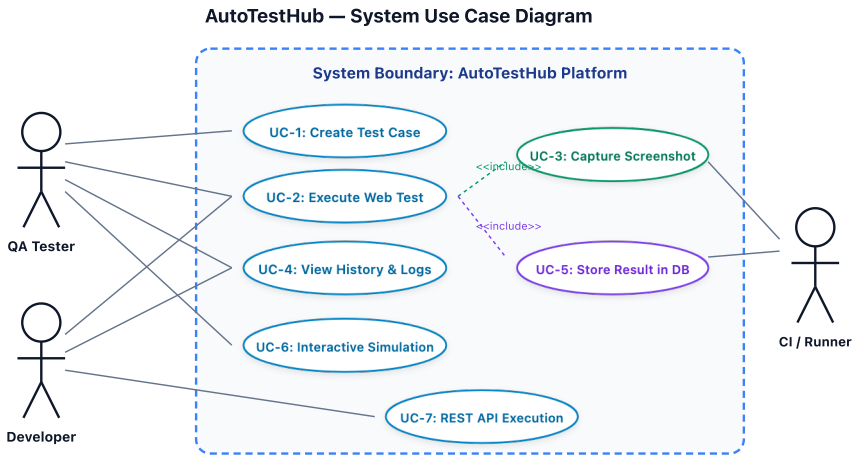


Figure 1: AutoTestHub System Use Case Model.

7.1 Use Case Specifications

Use Case	Primary Actor	Main Flow Summary	Postcondition
UC-01: Create Test	QA Tester	Sends JSON to `POST /api/tests`; backend validates and saves to DB.	TestCase saved with ID.
UC-02: Run Test	QA / Developer	Invokes `POST /api/tests/:id/run`; execution bridge spawns Python worker.	TestResult saved with screenshot.
UC-03: Control Desk	QA Tester	Selects task on UI, sets speed, views live execution in Chrome and log.	Run completed and logged.

7.2 Agile User Stories with Acceptance Criteria

Story 1 (Declarative Ingestion): As a QA Engineer, I want to define browser tests in JSON so that I can create regression suites without boilerplate code.
Criteria: Given a JSON test with ID, URL, and actions, when parsed, schema validation succeeds. If URL is `file://`, a ValueError is raised immediately.

Story 2 (Fail-Safe Recovery): As a DevOps Engineer, I want the automation worker to guarantee process termination so that orphaned Chrome processes do not leak host memory.
Criteria: Given an action failure (missing element), when timeout occurs, failure screenshot is captured, `driver.quit()` runs in finally block, and code 1 is returned.

8. System Design & Architecture

AutoTestHub employs a decoupled four-tier architecture isolating heavy synchronous browser driver operations from asynchronous web APIs.

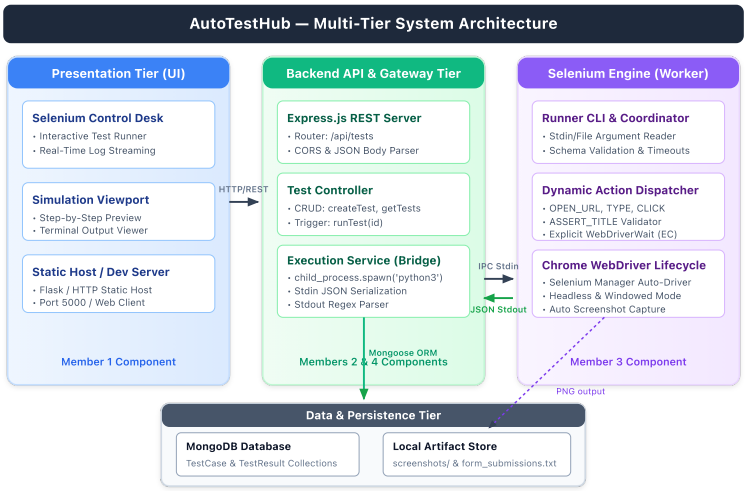


Figure 2: Multi-Tier System Architecture and Inter-Process Communication Flow.

8.1 Module Breakdown & Subsystem Responsibilities

- **Presentation Tier (`selenium_project/frontend/`):** Interactive Control Desk UI, speed slider, headless toggle, simulation viewport, and real-time terminal output viewer.
- **Backend Gateway Tier (`backend/`):** Node.js and Express REST services exposing endpoints (`POST /api/tests`, `GET /api/tests`, `POST /api/tests/:id/run`), CORS configuration, and body validation.
- **Execution Bridge (`backend/services/executionService.js`):** Spawns the Python child process, serializes JSON into stdin, extracts output JSON via regex, and saves `TestResult` documents to MongoDB.
- **Selenium Engine Tier (`selenium_worker/`):** Python 3.12 worker utilizing Selenium 4, Selenium Manager, dynamic action dispatchers, explicit waits, and automated screenshot generation.
- **Persistence Tier (MongoDB):** Stores test case definitions and historical test results.

8.2 Database Schema & Data Model

AutoTestHub uses MongoDB with Mongoose ODM to model test definitions and historical execution runs. A normalized 1-to-Many relationship connects test cases with their historical runs.

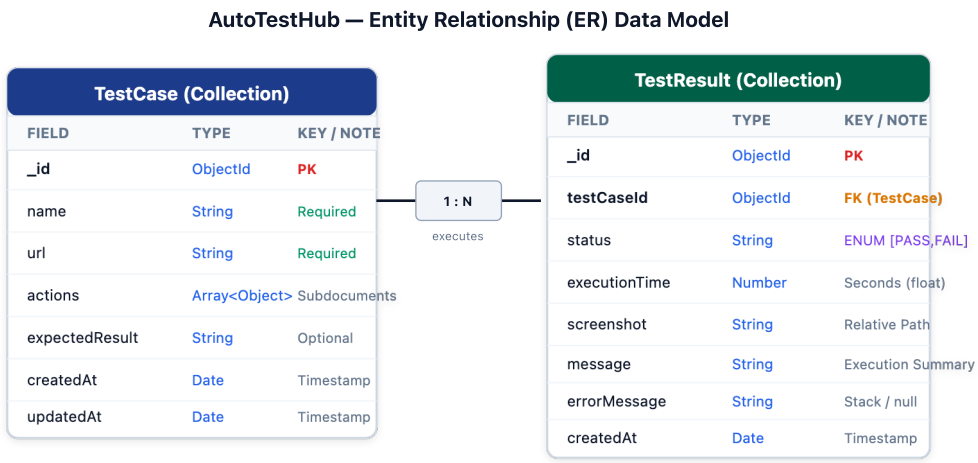


Figure 3: Entity-Relationship (ER) Data Model showing TestCase and TestResult collections.

Collection Schema Dictionaries

Collection	Field	Type	Constraint / Rule	Description
TestCase	`_id`	ObjectId	Primary Key	Auto-generated unique test identifier.
	`name`	String	Required, Trimmed	Descriptive name of the test scenario.
	`url`	String	Required	Target base URL for navigation.
	`actions`	Array<Object>	Array of subdocs	List of action objects (`type` , `locator` , etc.).
TestResult	`_id`	ObjectId	Primary Key	Auto-generated run identifier.
	`testCaseId`	ObjectId	Required, Ref: TestCase	Foreign key reference with index.
	`status`	String	Enum: [PASSED, FAILED]	Outcome verdict of the test run.
	`executionTime`	Number	Float (Seconds)	Total duration of browser execution.
	`screenshot`	String	Relative path	Filesystem path to captured PNG screenshot.

8.3 UML Behavioral & Structural Models

A. End-to-End Execution Sequence Diagram

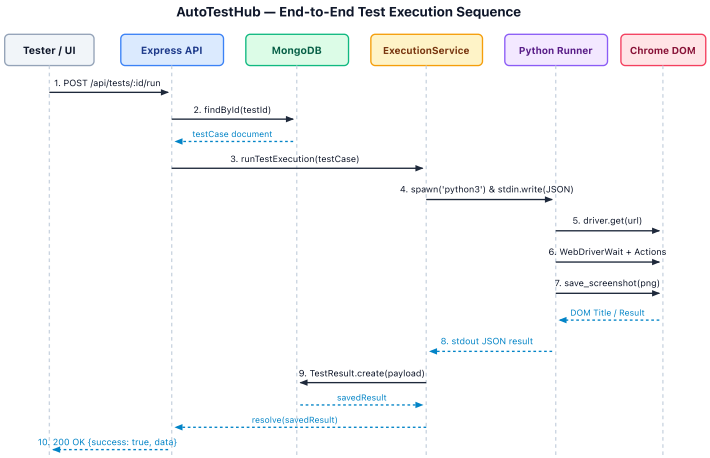


Figure 4: End-to-End Test Execution Sequence Diagram from UI Trigger to MongoDB Persistence.

B. System Class Diagram & Interfaces

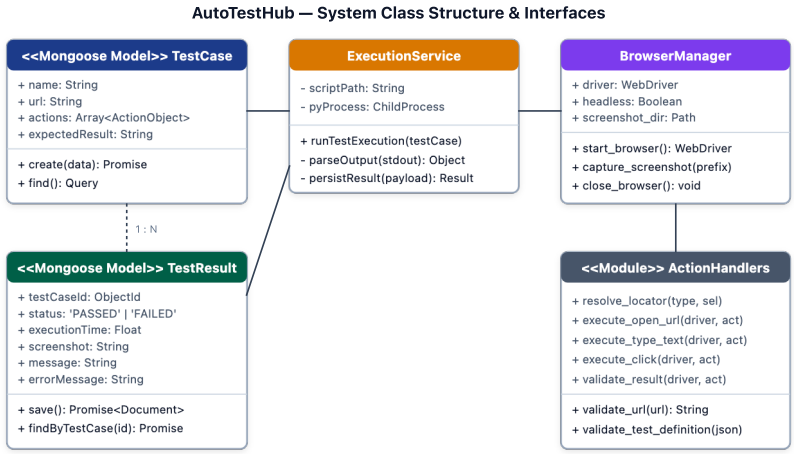


Figure 5: System Class Diagram depicting Node.js Mongoose models and Python Worker classes.

9. Technology Stack and Technical Justification

The technology stack was chosen to maximize developer velocity, eliminate external dependencies, and guarantee cross-platform execution.

Layer	Technology	Technical Justification & Rationale
Core Engine	**Python 3.12 • Selenium 4.x**	Built-in **Selenium Manager** resolves ChromeDriver automatically without third-party tools. Python offers fast execution and native test libraries.
API Gateway	**Node.js • Express.js 5.x**	Asynchronous non-blocking I/O event loop ideal for managing long-lived child process streams (stdin/stdout) without blocking HTTP worker threads.
Database & ODM	**MongoDB 7.x • Mongoose 9.x**	BSON document model stores polymorphic action definitions naturally without complex SQL joins. Mongoose enforces strict schema validation and indexes.
Presentation	**HTML5 • CSS3 • Vanilla JS**	Zero build step overhead. Provides instant load times, dark/light theme switching, and real-time terminal output streaming.
Test Suite	**Python `unittest` • Local HTTP**	In-memory tests run in <5ms. Built-in `http.server` enables integration tests against local sandboxes without internet access.

Comparison with Alternative Solutions

Dimension	AutoTestHub	Raw Selenium Scripts	Commercial SaaS (e.g., Cypress Cloud)
Driver Setup	Automatic (Selenium Manager)	Manual download / PATH config	Proprietary cloud agents
Test Definition	Declarative JSON schemas	Imperative Python/Java code	Custom JavaScript DSL
Cost & Privacy	100% Open & Localhost execution	Free / Localhost	Expensive monthly subscriptions
Evidence Auditing	Auto-saved PNGs & MongoDB logs	Manual custom scripting	Hosted cloud storage

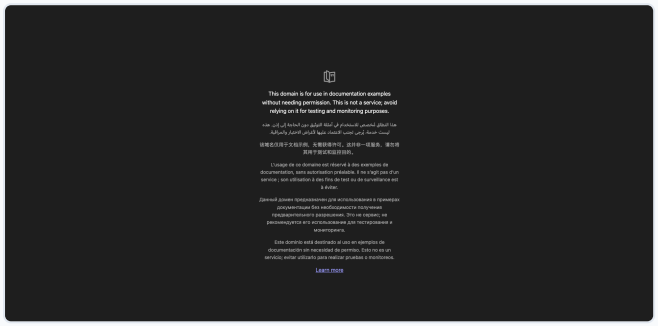
10. Implementation Progress

By the mid-semester evaluation, the project group has achieved **50% overall completion**, fulfilling all mid-semester requirements:

10.1 Completed vs. In-Progress Features

- **Completed:** End-to-end integration pipeline (Express → executionService → Python → Chrome → MongoDB), dynamic action handlers (`OPEN_URL`, `TYPE`, `CLICK`, `ASSERT_TITLE`), URL safety validator, auto-screenshot generation, and interactive Control Desk UI.
- **In Progress:** Distributed worker queue (Redis/BullMQ), full React UI migration, and advanced text assertions.

10.2 Screenshots of the Working System



Screenshot 1: Pipeline execution (Example.com).



Screenshot 2: Web form test execution.

11. Testing Performed & Verification Matrix

AutoTestHub was validated using 16 automated tests across unit and integration suites, achieving a **100% pass rate** in 18.5 seconds.

Test ID	Suite / Category	Input / Scenario	Expected Result	Actual Result	Status
`TC-01`	Unit: Validation	Valid JSON test definition	Validation passes (True)	Validation passed (0.00s)	PASS
`TC-02`	Unit: Validation	Missing `testCaseId` field	Rejection with field error	Rejected as expected	PASS
`TC-03`	Unit: Security	Insecure `file:///etc/passwd`	Raises ValueError	ValueError raised	PASS
`TC-04`	Unit: Locators	Resolve `css` selector	Maps to `By.CSS_SELECTOR`	Mapped accurately	PASS
`TC-05`	Int: Navigation	Navigate to sandbox URL	Status: PASSED	Status: PASSED (1.2s)	PASS
`TC-06`	Int: Assertion	Assert matching title	Status: PASSED, title matches	Matched 'Controlled Test'	PASS
`TC-07`	Int: Assertion	Assert mismatched title	Status: FAILED, mismatch error	Captured mismatch correctly	PASS
`TC-08`	Int: Form Flow	Open → Type → Click → Assert	Status: PASSED, title toggled	Form submitted, passed	PASS
`TC-09`	Int: Timeout	Click missing element (0.5s)	Status: FAILED, timeout msg	Timed out cleanly	PASS
`TC-10`	Int: Evidence	Successful test execution	PNG screenshot created on disk	PNG created and verified	PASS
`TC-11`	Int: Recovery	Run fail test then pass test	Pass test succeeds cleanly	Subsequent test passed	PASS
`TC-12`	API: Health	`GET /api/health`	HTTP 200 OK, running msg	HTTP 200 {"success": true}	PASS
`TC-13`	API: Create	`POST /api/tests`	HTTP 201 Created with _id	Document created in DB	PASS
`TC-14`	API: Pipeline	`POST /api/tests/:id/run`	Spawns worker, saves result	Result saved to MongoDB	PASS

Test Execution Evidence Log (Snippet)

```
Ran 16 tests in 18.552s • OK • [browser] Screenshot captured: screenshots/form-flow-001_20261019_162
{"testCaseId": "form-flow-001", "status": "PASSED", "executionTime": 1.86, "screenshot": "screenshot
```

12. Project-Management Status

12.1 Original Project Plan vs. Current Progress

The project plan was established for an 8-sprint (16-week) development lifecycle. The target for the mid-semester evaluation was 40%–60% completion. As demonstrated by the operational end-to-end pipeline, the project has achieved **50% completion** exactly on schedule.

Sprint Phase	Target Deliverables	Planned Target	Actual Progress	Status
Sprint 1	Requirements, WBS, Tech Stack, Architecture	10%	12%	ON TRACK
Sprint 2	Python Worker, Selenium Manager, MongoDB Schemas	25%	28%	ON TRACK
Sprint 3	Execution Service Bridge, Control Desk UI, Test Suites	45%–50%	50%	ACHIEVED
Sprints 4–8	Queue Worker, React Dashboard, CI/CD, Final Polish	100%	Remaining 50%	SCHEDULED

12.2 Delayed Tasks & Risk Mitigation Plan

Risk Description	Impact	Mitigation Strategy Executed
ChromeDriver incompatibility	High	Integrated Selenium 4 Selenium Manager; driver auto-resolved at runtime.
Asynchronous DOM flakiness	High	Implemented explicit `WebDriverWait` with ExpectedConditions on all actions.
Zombie process memory leak	High	Enforced strict `try-finally` blocks with guaranteed `driver.quit()`.
Node.js JSON parse failures	Medium	Applied regex extractor (`/\{[\s\S]*\}/`) on stdout, isolating JSON from logs.

13. Individual Contribution Table

Group Member	Assigned Subsystem	Key Deliverables & Technical Contributions	Approx. LOC	Weight
Shristi Neekhra	Integration Lead & Execution Service	Developed `executionService.js` subprocess bridge, stdin/stdout IPC streams, standalone test pipeline (`testExecution.js`), and mid-semester release packaging.	~450 LOC	**25%**
Shifa Siddiqui	Backend API & Database Lead	Initialized Express server, MongoDB connection, Mongoose `TestCase` & `TestResult` schemas, REST routes (`POST /api/tests` , `GET /api/tests`), and error handling.	~400 LOC	**25%**
Shruti Gupta	Selenium Engine Architect	Architected Python worker module, runner CLI, Selenium Manager integration, dynamic action handlers (Open, Type, Click, Assert), in-memory tests, and integration sandbox.	~650 LOC	**25%**
Shriya Agarwal	Frontend Lead & Control Desk	Designed responsive Control Desk interface (HTML5/CSS3), real-time terminal streaming visualizer, simulation playback mode, speed controls, and theme toggling.	~450 LOC	**25%**

14. Challenges Faced and Solutions Attempted

1. **Challenge: Node.js JSON Parsing Interrupted by Third-Party Driver Logs**
   Problem: ChromeDriver logs printed to stdout corrupted Node's `JSON.parse()`.
   Solution: Redirected diagnostic logs to `stderr`, and applied multiline regex (`/\{[\s\S]*\}/`) in Node.js to extract JSON reliably.
2. **Challenge: Asynchronous Element Rendering Flakiness**
   Problem: Elements looked up immediately after navigation threw element not found errors.
   Solution: Replaced static sleep calls with explicit `WebDriverWait` with `visibility_of_element_located`.
3. **Challenge: Preventing Orphaned Background Browser Instances**
   Problem: Unexpected Python exceptions caused script to abort before calling `driver.quit()`.
   Solution: Wrapped all WebDriver operations inside a top-level `try–finally` block that guarantees driver cleanup.

15. Work Planned for the Remaining Semester

Following the mid-semester evaluation, the group will proceed through Sprints 4 to 8 to complete the remaining 50% of system scope:

- **Sprint 4 (Weeks 7–8): Advanced Assertions & Custom Timeouts:** Implement assertions for inner text matching, attribute value checks, and element presence, with per-action timeout overrides.
- **Sprint 5 (Weeks 9–10): React Test Builder Dashboard:** Develop a React SPA enabling visual test creation, drag-and-drop step reordering, and execution metrics visualization.
- **Sprint 6 (Weeks 11–12): Distributed Worker Queue:** Implement Redis and BullMQ to support concurrent multi-worker execution across multiple headless browser instances.
- **Sprint 7 (Weeks 13–14): CI/CD Integration & Webhooks:** Build GitHub Actions integration and webhook triggers for automated PR test runs.
- **Sprint 8 (Weeks 15–16): Cross-Browser Testing & Hardening:** Extend Selenium Manager to support Firefox and Edge; conduct stress testing, performance optimization, and final defense preparation.

16. References & Academic Integrity AI Disclosure

Academic References

1. IEEE Computer Society, "IEEE Recommended Practice for Software Requirements Specifications," *IEEE Std 830-1998*.
2. Selenium Project Documentation, "Selenium WebDriver & Selenium Manager Architecture," *SeleniumHQ*, 2024. Available: https://www.selenium.dev/documentation/
3. MongoDB Inc., "MongoDB Manual: Schema Validation and Document Design Patterns," 2024. Available: https://www.mongodb.com/docs/
4. Express.js Foundation, "Express Routing, Middleware, and Error Handling Guide," 2024. Available: https://expressjs.com/
5. Martin Fowler, "Continuous Integration and Automated Testing Strategies," *Addison-Wesley*, 2018.

Academic Integrity & Generative AI Tools Disclosure (Section 6 Compliance)

Tool Name	Purpose of Use	Nature of Generated Content	Verification / Modification by Students
Gemini / Antigravity	Documentation structuring, SVG diagram path boilerplate, and edge-case unit test brainstorming.	Markdown layouts, SVG vector coordinate scaffolding, and mock test JSON templates.	All code, architectures, schemas, and test cases were manually implemented, debugged against real Chrome sessions, verified in terminal runs, and thoroughly understood by all group members.

Mid-Semester Submission Verification Statement:
The undersigned students confirm that all submitted materials represent authentic group progress. At least one complete end-to-end feature is fully operational and demonstrated. No confidential credentials or API keys have been committed.